

Criterios cobertura de grafos: código fuente

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez (Versión en castellano)

Resumen

Una aplicación habitual de los criterios de grafos es al código fuente.

Grafo: Habitualmente, el grafo de control de flujo (GCF).

Cobertura de nodos: Ejecutar cada instrucción.

Cobertura de aristas: Ejecutar cada decisión (por verdadero y por falso).

Ciclos: Estructuras repetitivas

Grafos de control de flujo

Un **GCF** modela todas las ejecuciones de un programa mediante la descripción de sus estructuras de control.

Nodos: Instrucciones o secuencias de instrucciones (bloques básicos).

Aristas: Transfieren el control.

Bloque básico: Secuencia de instrucciones tal que si se ejecuta la primera entonces se ejecutan todas las demás (no hay ramificación).

Algunas veces se anotan los CFGs con información adicional:

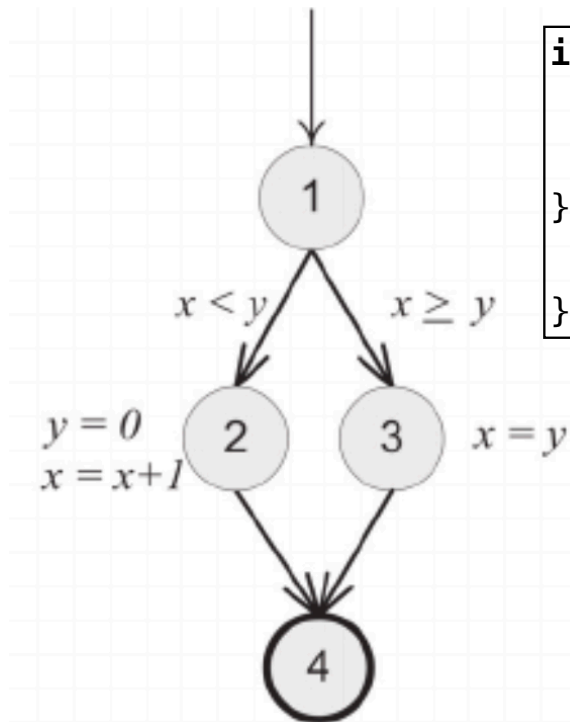
- Predicados en las ramas.

- Definición de variables.

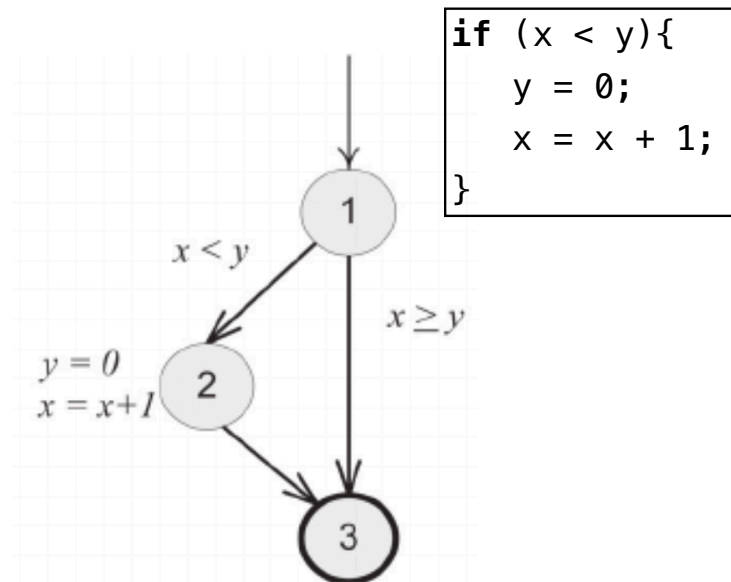
- Uso de variables.

A continuación veremos como se **traduce** de instrucciones a grafos.

GCF: la instrucción if



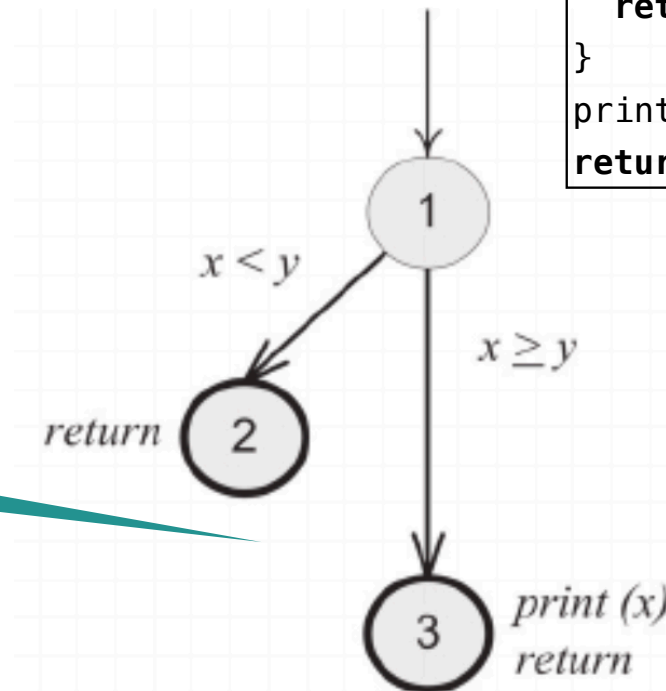
```
if (x < y){  
    y = 0;  
    x = x + 1;  
}else{  
    x = y;  
}
```



```
if (x < y){  
    y = 0;  
    x = x + 1;  
}
```

GCF: la instrucción if-return

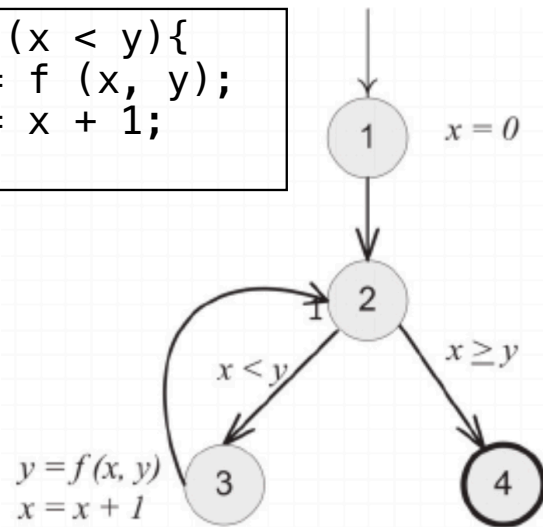
```
if (x < y){  
    return;  
}  
print(x)  
return;
```



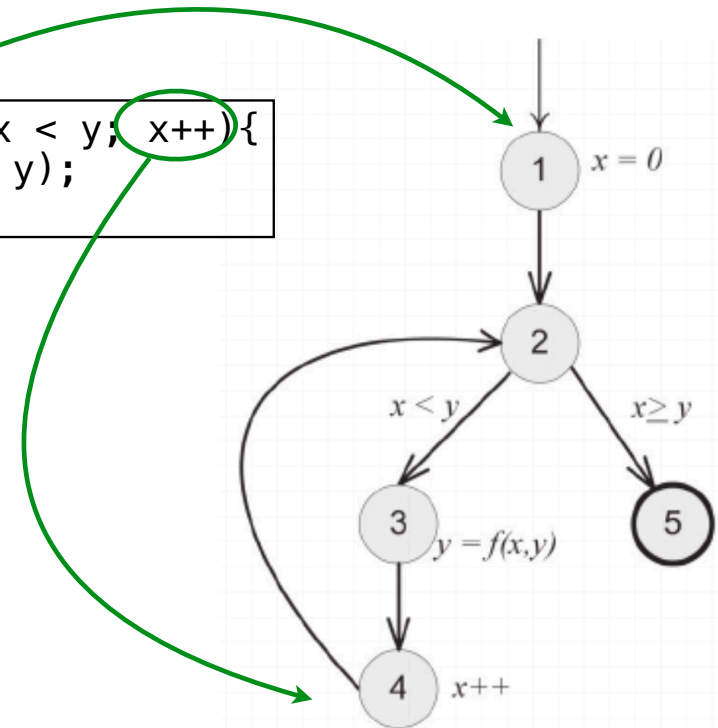
No hay arista entre 2 y 3!

GCF: Ciclos while y for

```
while (x < y){  
    y = f (x, y);  
    x = x + 1;  
}
```

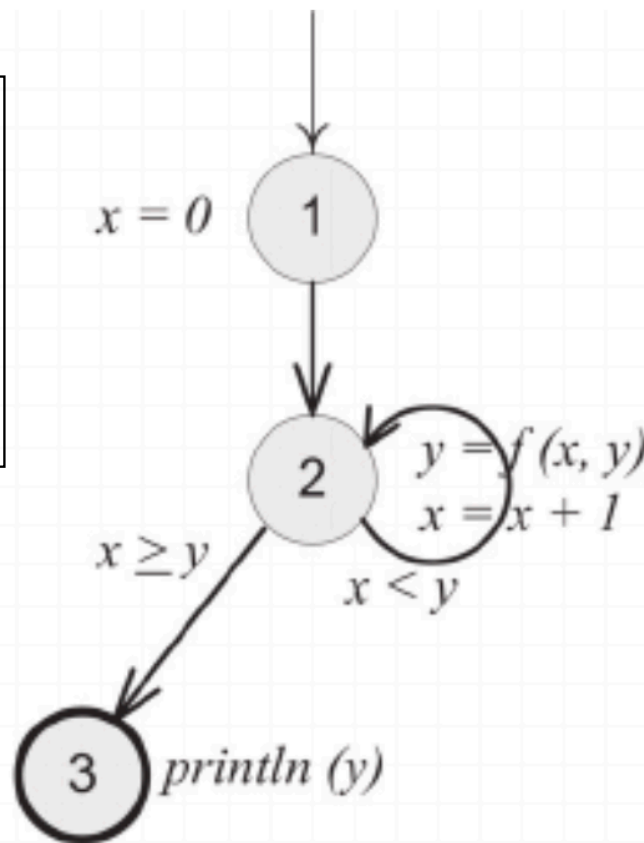


```
for (x = 0; x < y; x++){  
    y = f (x, y);  
}
```



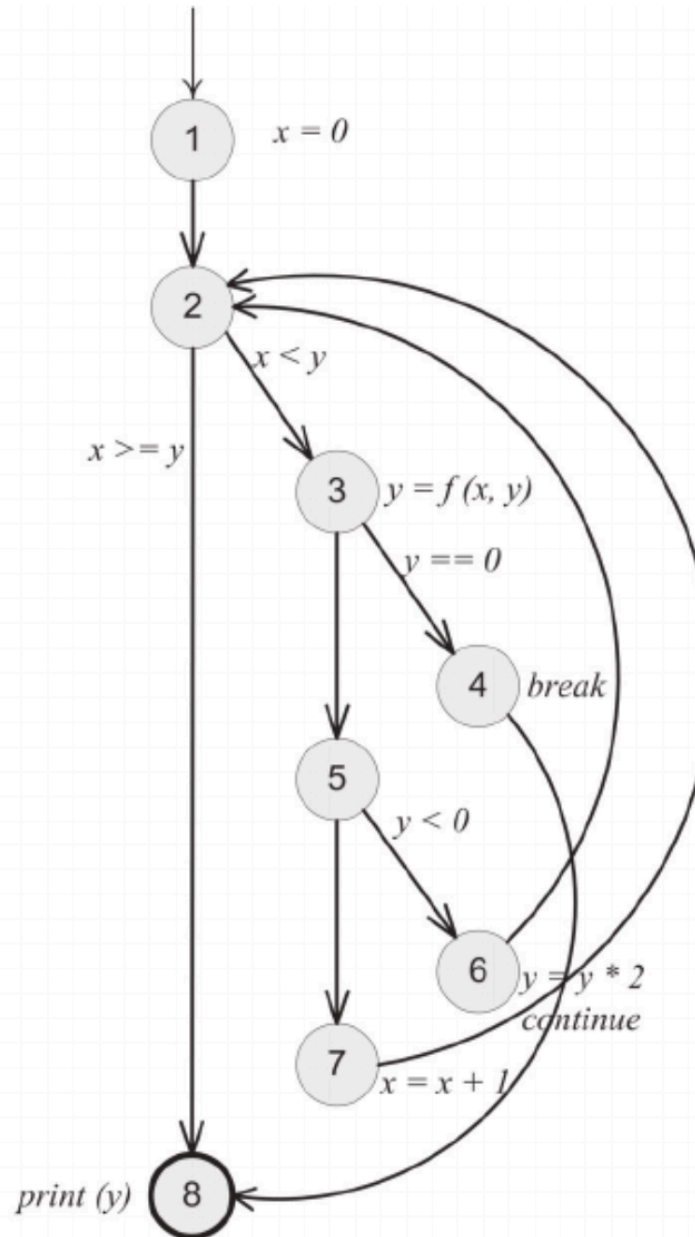
GCF: Ciclo do

```
x = 0;  
do{  
    y = f (x, y);  
    x = x + 1;  
} while (x < y);  
println (y)
```



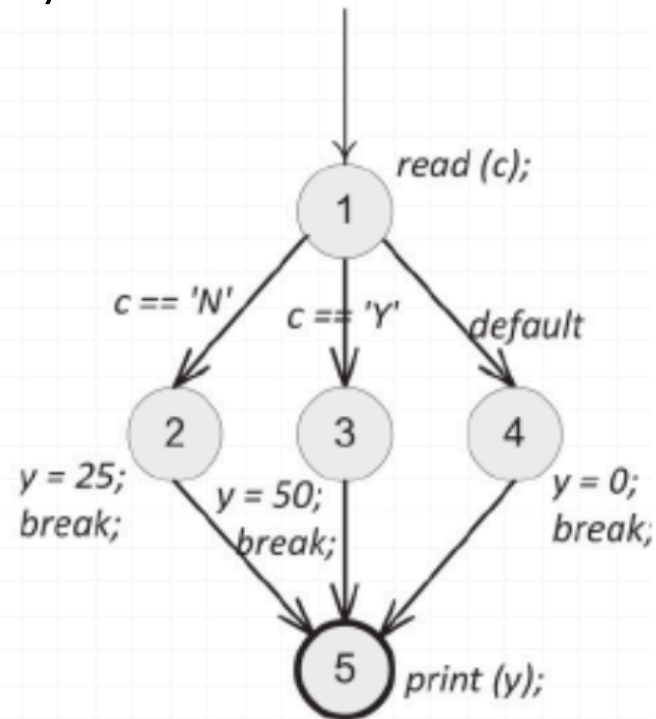
GCF: break, continue

```
x = 0;
while (x < y){
    y = f (x, y);
    if (y == 0){
        break;
    } else if (y < 0){
        y = y*2;
        continue;
    }
    x = x + 1;
}
print (y);
```



GCF: Case/switch

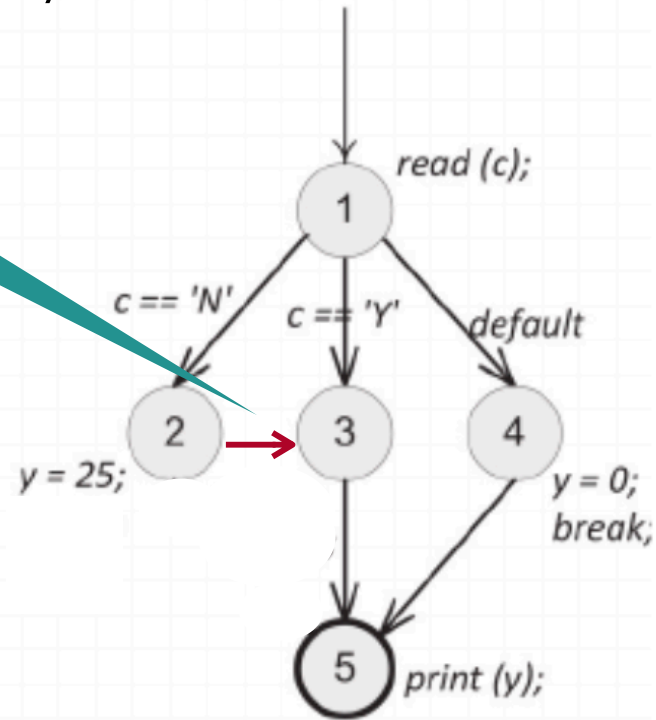
```
read (c) ;  
switch (c){  
  case 'N':  
    z = 25;  
    break;  
  case 'Y':  
    x = 50;  
    break;  
  default:  
    x = 0;  
    break;  
}  
print (x);
```



GCF: Case/switch

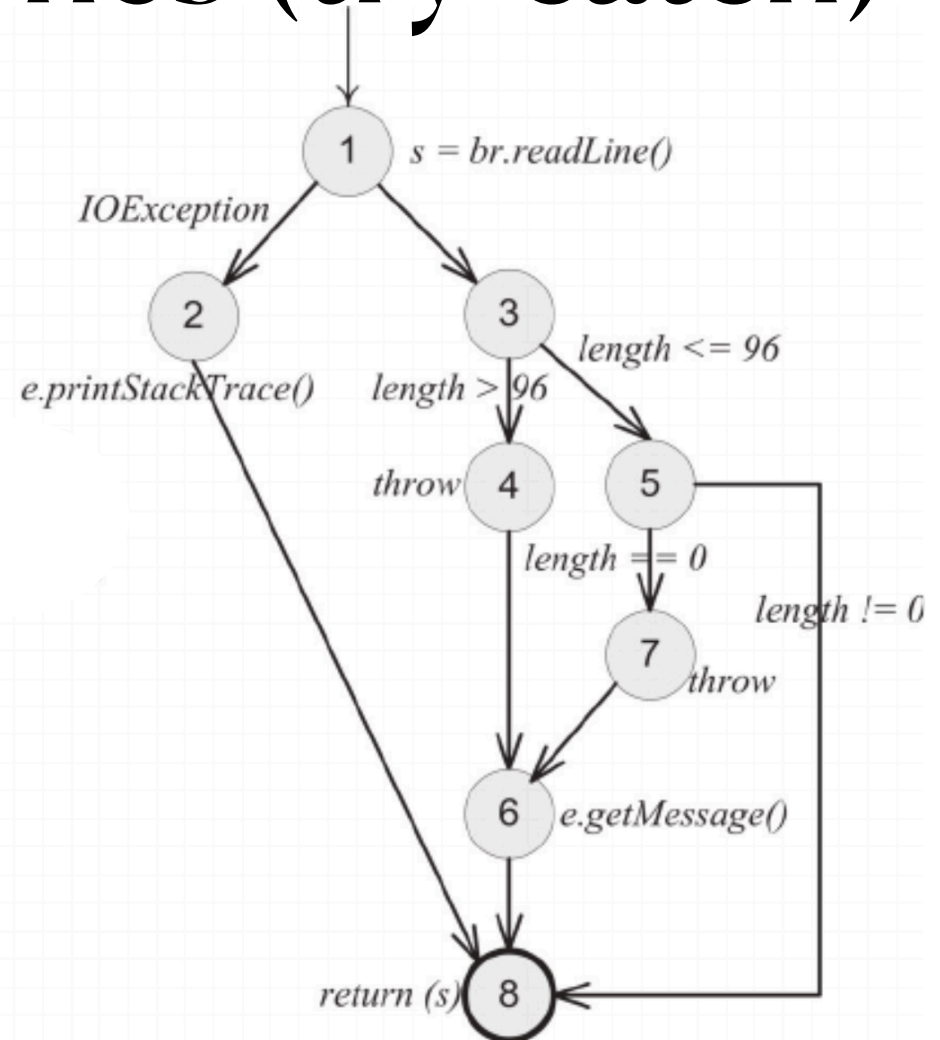
```
read (c) ;  
switch (c){  
  case 'N':  
    z = 25;  
  case 'Y':  
    x = 50;  
    break;  
  default:  
    x = 0;  
    break;  
}  
print (x);
```

No hay break



GCF: Excepciones (try-catch)

```
try
{
    s = br.readLine();
    if (s.length() > 96)
        throw new Exception("too long");
    if (s.length() == 0)
        throw new Exception("too short");
} (catch IOException e) {
    e.printStackTrace();
} (catch Exception e) {
    e.getMessage();
}
return (s);
```



Criterios de Cobertura aplicado a GCF

La aplicación de los criterios de cobertura vistos es directa, solo cambian los nombres:

Cobertura de nodos: Cobertura de Sentencia.

Cobertura de aristas: Cobertura de ramas (branches)

Ejemplo

```
public static void computeStats (int [ ] numbers){
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;

    sum = 0;
    for (int i = 0; i < length; i++){
        sum += numbers [ i ];
    }
    med  = numbers [ length / 2];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++){
        varsum = varsum + ((numbers [ I ] - mean) * (numbers [ I ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length: " + length);
    System.out.println ("mean: " + mean);
    System.out.println ("median: " + med);
    System.out.println ("variance: " + var);
    System.out.println ("standard deviation: " + sd);
}
```



Dibujar el GCF

```

public static void computeStats (int [ ] numbers){
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;
    sum = 0;

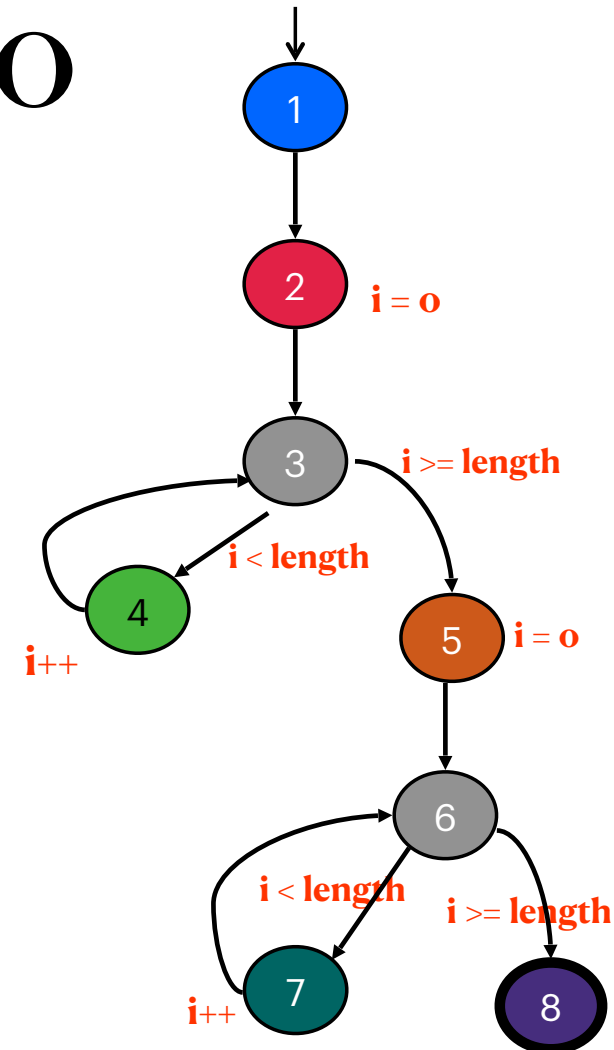
    for (int i = 0; i < length; i++){
        sum += numbers [ i ];
    }
    med  = numbers [ length / 2];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++){
        varsum = varsum + ((numbers [ i ] - mean) * (numbers [ i ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length: " + length);
    System.out.println ("mean: " + mean);
    System.out.println ("median: " + med);
    System.out.println ("variance: " + var);
    System.out.println ("standard deviation: " + sd);
}

```

Ejemplo



Requisitos de test y caminos de test: EC

Edge Coverage

Requisitos de test

A. [1, 2]

B. [2, 3]

C. [3, 4]

D. [3, 5]

E. [4, 3]

F. [5, 6]

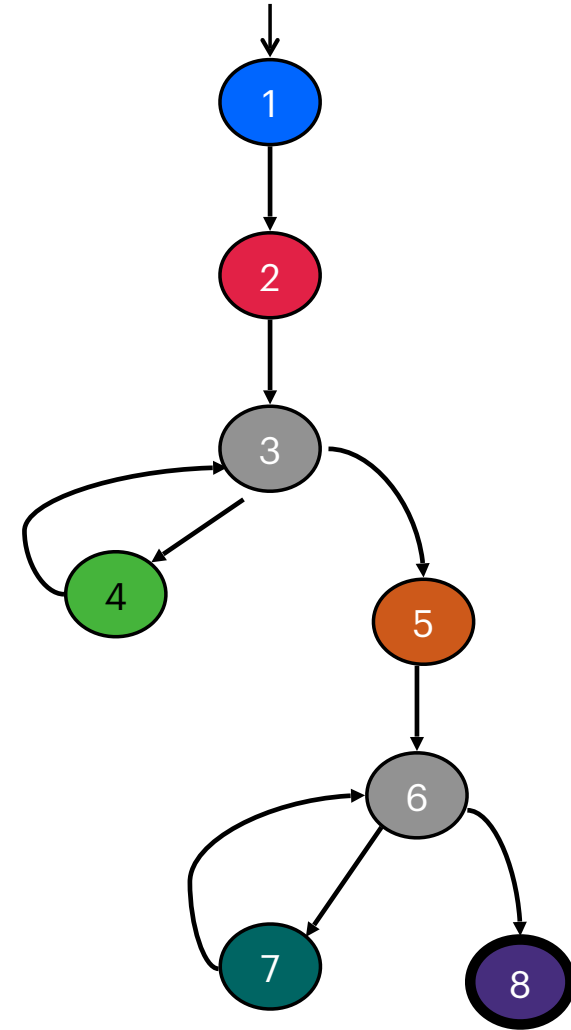
G. [6, 7]

H. [6, 8]

I. [7, 6]

Caminos de test

[1, 2, 3, 4, 3, 5, 6, 7, 6, 8]



Requisitos de test y caminos de test: EPC

Edge-Pair Coverage

Requisitos de test

A. [1, 2, 3]

B. [2, 3, 4]

C. [2, 3, 5]

D. [3, 4, 3]

E. [3, 5, 6]

F. [4, 3, 5]

G. [5, 6, 7]

H. [5, 6, 8]

I. [6, 7, 6]

J. [7, 6, 8]

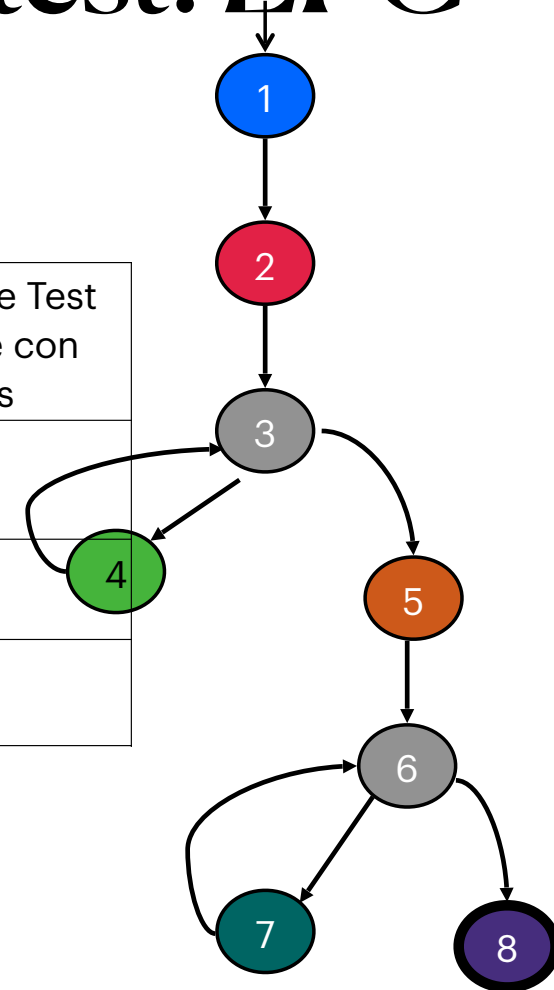
K. [4, 3, 4]

L. [7, 6, 7]

Camino de Test	Requisitos de Test que cubre (recorre/tours)	Requisitos de Test que recorre con sidetrips
[1, 2, 3, 4, 3, 5, 6, 7, 6, 8]	A, B, D, E, F, G, I, J	C, H
[1, 2, 3, 5, 6, 8]	A, C, E, H	
[1, 2, 3, 4, 3, 4, 3, 5, 6, 7, 6, 6, 8]	A, B, D, E, F, G, I, J, K, L	C, H

Redundante! Un conjunto Minimal es más barato

[1, 2, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 8]



Requisitos de test y caminos de test: PPC

Prime Path Coverage

Requisitos de
test

A. [3, 4, 3]

B. [4, 3, 4]

C. [7, 6, 7]

D. [7, 6, 8]

E. [6, 7, 6]

F. [1, 2, 3, 4]

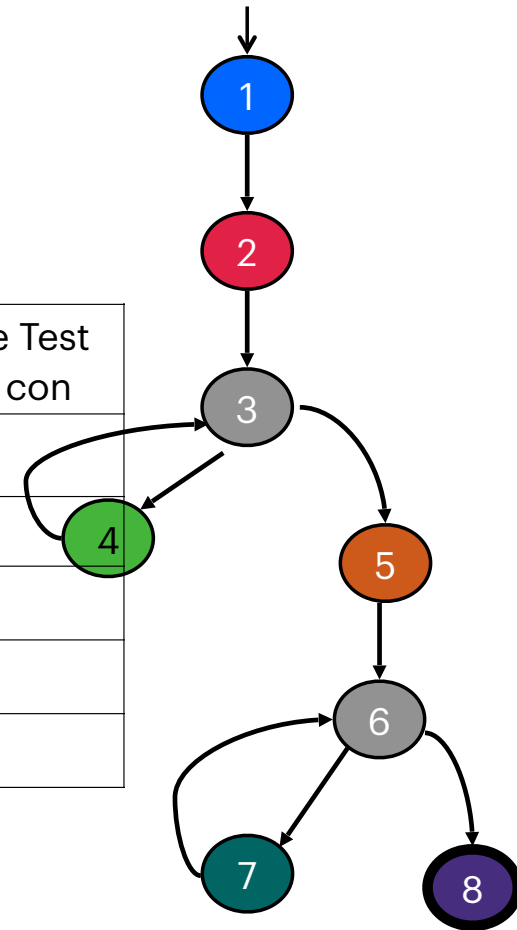
G. [4, 3, 5, 6, 7]

H. [4, 3, 5, 6, 8]

I. [1, 2, 3, 5, 6, 7]

J. [1, 2, 3, 5, 6, 8]

Camino de Test	Requisitos de Test que cubre (recorre/ que recorre con	Requisitos de Test que recorre con
[1, 2, 3, 4, 3, 5, 6, 7, 6, 8]	A, D, E, F, G	H, I, J
[1, 2, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 8]	A, B, C, D, E, F, G	H, I, J
[1, 2, 3, 4, 3, 5, 6, 8]	A, F, H	J
[1, 2, 3, 5, 6, 7, 6, 8]	D, E, F, I	J
[1, 2, 3, 5, 6, 8]	J	



Resumen

La aplicación de los criterios de test a los grafos de control de flujo es relativamente sencilla.

Aún así, hay que tomar algunas decisiones sutiles al traducir de estructuras de control a grafos.

Algunas herramientas asignan un nodo a cada instrucción (en lugar de asignarlo a cada bloque básico).

Resumen

Los grafos no solo pueden ser obtenidos a partir del código fuente, por ejemplo es posible obtenerlos de elementos de diseño

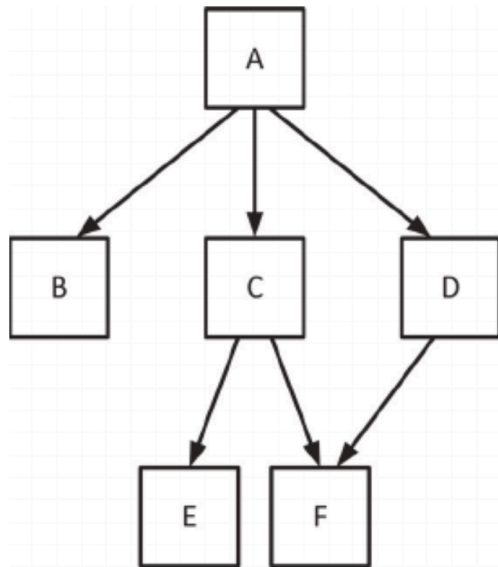
En Orientación a objetos:

El énfasis en la modularidad y la reutilización lleva a que la complejidad se desplace a las relaciones/conexiones entre las partes del sistema.

Por ello, testear estas relaciones es más importante que antes.

Los grafos se basan en las conexiones entre las componentes software.

Grafo de llamadas



Es el tipo de grafo más común para testear el diseño estructural.

Nodos: Unidades (en Java, métodos).

Aristas: Llamadas a unidades.

Node coverage: Llamar a cada unidad al menos una vez (cobertura de métodos).

Edge coverage: Ejecutar cada llamada al menos una vez (cobertura de llamadas).